

# 琥珀 —— 会自己动手把事情办完的私人助理

作品设计说明 · 人工智能实用智能系统（软件程序类）

参赛作者：\_\_\_\_\_ 年级：\_\_\_\_\_

学校：\_\_\_\_\_ 指导教师：\_\_\_\_\_

完成日期：2026 年 \_\_\_\_ 月 \_\_\_\_ 日 演示方式：现场真机演示（手机 / 电脑浏览器）

## 摘要

我做的作品叫「琥珀」，是一个**会自己动手把事情办完的私人助理**。用户对着它说话，它不只是聊天：它会真的去查资料、写代码、做计算，做完主动回来告诉用户结果；它记得住以前说过的事；出了错会如实说，半途被打断也能接着说。它最特别的地方是**它能给用户"造软件"**：用户说一句「帮我做一个记账的小程序」，它当场写出程序，桌面上当场长出一个图标，点开就能用、数据还存得住。做它的起点是一个很普通的发现：**人跟 AI 打交道，卡住的往往不是想法，而是"输入"**——想清楚一件事只要一瞬，把它一个字一个字敲进去却要很久，还要先学会打字。所以琥珀要做的，是把**"输入"这一关从键盘换成说话**：会说话，就能用。这样一来它的使用者就不只是"会打字的人"：**八九岁的孩子说得清就能用，六七十岁的老人不用学拼音也能用**，在手机和平板上移动办公的人也能一句话把事交出去。为了让跨度这么大的使用者都能用得起来，整个界面守着三条硬规矩：字放到 3 倍不破版、每个按钮的可点范围不小于 44 像素、不允许出现只有图形没有文字、按不动的按钮。系统用「一人一台专属小主机」的方式保证隐私：每个人的数据、钥匙、助手都跑在自己那台隔离的小主机里。目前的版本（V1.0）已经在真实的服务器和真机上运行（三台用户的小主机在跑），1513 条自动判据全部通过。

## 一、作品是做什么的

打开琥珀，看到的是两样东西：一个**聊天窗口**，和一个**桌面**。桌面上一格图标 = 一个小程序 = 它和用户之间的一条对话。用户可以打字，也可以按住麦克风说话。

**它给用户的一句承诺：你说的事它真会去做，不只是陪聊。**

这句话拆成三条能验证的行为：

- **记得住**：关掉页面、甚至整台机器重启，它还记得之前说过的事（聊天记录、做过的事、答应过的事都落了账）。
- **真去做**：它能读文件、跑命令、查网、算数、画图、记账，还能**写一个新的小程序出来**。

- **做完回来说：**长一点的活它先放到后台，一句"我先去办"；办完或者失败，它主动说一声；用户随时可以问「那件怎么样了」。



图 1 真实运行中的桌面（线上截图 · 取自一个真实用户账号，不是开发者的演示环境；这台设备设了壁纸）。每一格是一个小程序，也就是一条对话；左下角「+」是"造一个新的"。

## 二、为什么要做它

### 2.1 一个观察：跟 AI 打交道，卡点在"输入"

这几年我用 AI 的方式变了：以前要坐到电脑前，现在越来越多是在手机上、平板上 —— 走到哪儿都能办点事。可越是这样，我越发现一件事：**我说得比打得快得多**。脑子里想清楚一件事只要一瞬，把它一个字一个字敲进输入框却要花好一会儿，还得先会打字、打准、打对。**跟 AI 打交道，卡住的常常不是"想法"，而是"输入"这一关**。而说话是人天生就会的：孩子先会说话，很多年后才学会写字；老人打不了拼音，但话说得很清楚。

所以琥珀要做的不是"又一个聊天软件"，而是把"输入"这一关从键盘换成说话：会说话，就能用。

### 2.2 所以它的使用者很宽：从八九岁的孩子到六七十岁的老人

| 谁               | 他在这儿是怎么用的                                   |
|-----------------|---------------------------------------------|
| 八九岁的孩子          | 还没学打字（或者打得很慢），但话说得清楚：想看一个自己出题的算术小游戏，说一句话就有了 |
| 六七十岁的老人         | 不会拼音、看不清小字：不用学输入法，按住说一句就行；字还能放大到 3 倍        |
| 在手机 / 平板上移动办公的人 | 手上正忙、走在路上、只有手机：一句话把事交出去，长活它自己在后台办完再来报       |

|          |                                     |
|----------|-------------------------------------|
| 不想学新软件的人 | 没有菜单要翻、没有行话要懂 —— 界面上只有"说一句"和"它做的东西" |
|----------|-------------------------------------|

一句话：它的使用者从"会打字的人"扩到了"会说话的人"。

2.3 光有"语音输入"还不够：听清 ≠ 理解对

如果只是把说的话转成文字、直接丢给 AI，会出事：**转文字会错**（同音字、断句、说到一半），AI 却会照着这个错的理解真的去做。对说得不完整的孩子、带口音的老人、环境嘈杂的场合，这个问题更明显。所以琥珀在"听"和"做"之间加了一层**只负责听懂**的快速模型：它不确定就问一句（同时念出来 + 文字显示），问明白了才发；问不明白就不硬发。这也是"宽人群都能用"的必要条件 —— 不是每个人都能把话说得像打字稿一样清楚。

2.4 设计给"最难的那一端"，两端都受益

使用者跨度这么大，界面就不能只照顾"熟练用户"。日常里最容易出问题的三件事，本项目当成**缺陷**来管（而不是当成风格问题），因为它们在两端都致命：

- 文字放大到**3 倍**时界面不许破版、不许溢出（有专门的自动检查，五档字号全跑一遍）；
- 每一个可以点的东西，可点范围**不小于 44 像素**（有检查逐屏扫描）；
- 界面上**不许出现内部词**（"工作区 / 云端 / 客户端 / 模型"这一类有一份禁用词表，代码里每一句给用户看的话都要过这道检查）。

"看不清小字的人""不会打字的人"是我们的**设计基准**（最难的那一端能做到，别的人一定也能用），而**不是**使用者的范围。

2.5 现在的 AI 助手差在哪

| 常见问题                  | 琥珀怎么处理                                                           |
|-----------------------|------------------------------------------------------------------|
| 只会说，不会做；说得头头是道，事情没办   | 助手手里有真的工具（读文件、跑命令、查网、造小程序），并且 <b>每一轮活都落账</b> ：做了什么、成没成、花了多少，都查得到 |
| 做完不说，用户不知道到底办没办       | 收口时主动说结果；失败也主动说清是哪一步没成，而不是悄悄卡住                                   |
| 中途被打断就"失忆"，或者干脆假装没发生过 | 它把"还开着的活"逐件记在盘上；重启后会逐件如实回答"做完了 / 已停"，不会变成"查无此事"                  |
| 用户不敢把隐私交给它            | 一人一台隔离的小主机，钥匙只在自己那台里，连助手自己都读不到明文钥匙                               |

### 三、技术原理

#### 3.1 总体结构

系统由三部分组成：用户手上的**界面**、一台负责转发与记账的**中心服务**，以及**每个人自己的那台小主机**（用容器做的隔离环境，我们叫它"盒子"）。真正干活的东西（助手、聊天记录、小程序、钥匙）全都在用户自己的盒子里。

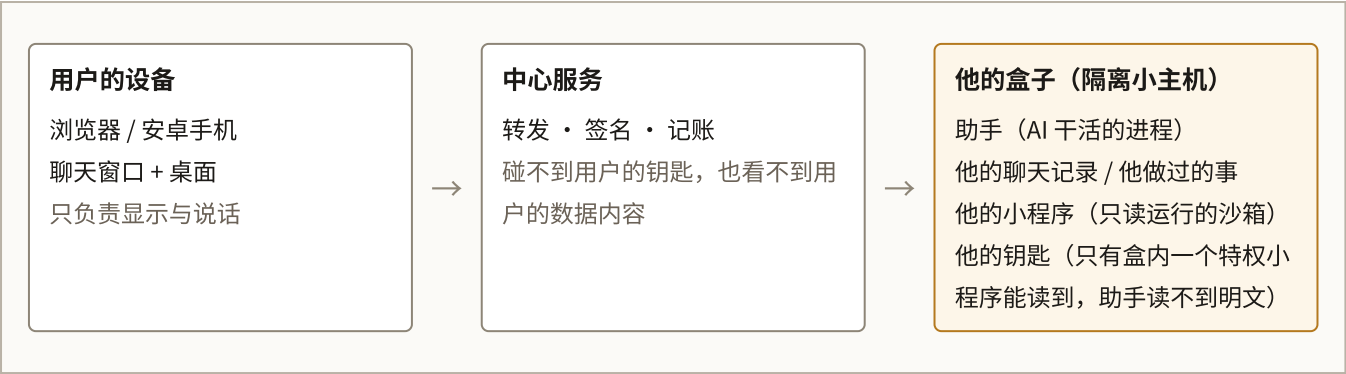


图2 系统结构：一人一台盒子，盒子之间互相看不见。

#### 3.2 一轮对话：一条有始有终的流水线

用户说一句话，在系统里不是"丢给大模型答一句"，而是变成**一件有票、有账、有结局的活**：

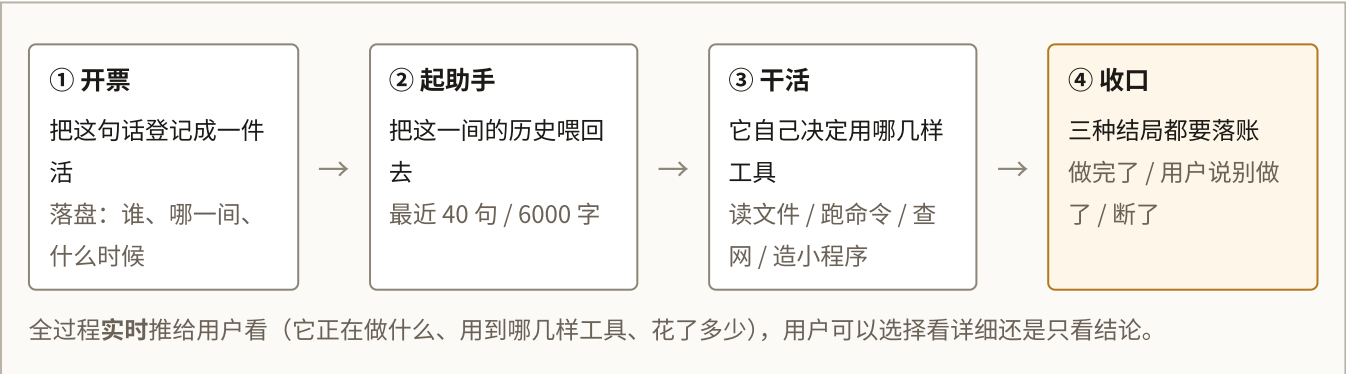


图3 一轮对话的流水线。"有始有终"是整个系统的地基：没有收口的活，重启后要能逐件如实交代。

#### 3.3 人工智能用在哪三处

##### （1）当"大脑和手"：由大模型驱动的工具调用

助手背后是一个大语言模型（低推理开销那一档，响应快）。它做的不是"续写文字"，而是**决定下一步用哪样工具、看结果、再决定下一步**，直到这件事办完。这些工具的调用与结果都进聊天窗口的记录里——也就是说，**用户能看见 AI 是怎么一步步做事的**，不是黑箱里冒出一个答案。

##### （2）当"程序员"：由 AI 写出来的小程序，跑在沙箱里

这是本作品最有意思的一段。用户说「帮我做一个记账的小程序」，系统做了四件事：

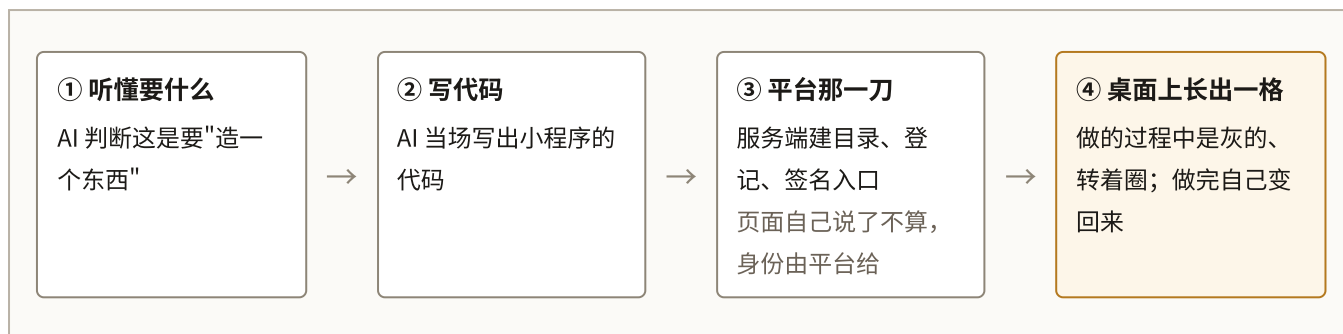


图4 一个小程序从"一句话"变成"桌面上的一格"。

每个小程序都跑在一个关起来的沙箱里：代码是**只读**挂载进去的；每个小程序有自己的一格数据库，**别的程序读不到**；它想联网、想连哪个网站、想用 AI，都要用户**第一次打开时点过头**才生效（默认什么都没有）。它上网还会留痕：去过哪些域名、多少次，都记在账上（不记录内容）。

### （3）当"耳朵和嘴"：听得见，还要听得懂（在研）

现在已经做到的：按住麦克风说话，语音**流式识别**成文字（用的是豆包大模型语音识别），说完文字就在输入框里。正在做的下一版（V2.0 第一件）解决的是一件更细的事：**听清楚了，不等于理解对了**。所以我们加一层"只听懂、不创作"的快速模型：

- 它把用户说的话理顺（补标点、改同音错字），**但不许添加用户没提过的要求**；
- 它只回答"我哪里不确定"（是上周还是上个月？给谁看？要不要存数据？）；
- 不确定就**问回去**（同时用语音念出来 + 文字流式显示），问明白了才自动发出去；
- 问了两轮还不清楚，就不硬发 —— 屏幕上如实说"这句我没听清，你再说一遍"。

这一层的关键设计是"宁可问一句，也不替用户猜"：因为猜错以后，助手会真的照错的去做。

## 3.4 怎么做到"你的东西只有你有"

- **一人一台盒子**：每个用户一个独立容器（无特权容器 + 去掉全部特权能力后再按需加回），代码层只读挂载，每台限制内存（现在每台 4 GiB）；一台出问题不影响别人。
- **钥匙不出盒子**：模型钥匙放在盒子里的一个特权小程序手里，干活的助手环境里只有一个占位符 —— 它连"读钥匙"这条路都没有。
- **删号即删盒**：账号不要了，那台盒子一起删掉。

## 四、实现过程

我用了一套"先立规矩、再写代码，而且每一步都要能被推翻"的做法：

- 1. **先写"要什么、为什么"**：把设计决定一条条写下来（为什么这么做、代价是什么、以后不许怎么做），前后做了两轮独立评审。这些文档是"规矩"，不是"事后说明"。
- 2. **判据先行**：每一件功能都配"怎么算做完了"的自动判据；代码写完，判据必须通过。
- 3. **负向对照 + 变异验证**：光"通过"不算。每条判据都要证明它**抓得住错**——我会故意把那条规矩删掉或改坏，看判据是不是当场变红；不红就说明这条判据是空转的。
- 4. **每一步留账**：做了一批就记一行账（做了什么、读数是多少、还欠什么）。现在仓库里有 **182 篇实现契约**，全部可追溯。
- 5. **边做边真上线**：目前版本 **V1.0**（2026-10-04 定档），已经部署在真实的服务器上、真机上在用（现在有三台用户的小主机在跑）；之后只修严重问题，新功能进 V2.0。

这一段最想说明的是：**这个作品不是"演示原型"**。它有真用户、真服务器、真容器，也有真出过的故障和修法（例如有一次浏览器端因为地址算法写错导致数据流被浏览器拦掉，而当时所有自动判据都是绿的——因为判据绕开了出错的那一行。我们把这条教训写成了新规矩：**凡是客户端自己算出来的东西，检查就必须打在客户端这一侧**）。

## 五、测试结果

| 检查            | 结果（2026-10-04 实测）                                                                                     |
|---------------|-------------------------------------------------------------------------------------------------------|
| 服务端自动判据       | <b>1513 条全部通过，0 条失败</b> （分布在 142 个判据文件里）；其中不少是端到端判据：真起服务、真连接、真开容器                                     |
| 客户端自动判据       | 静态分析 <b>0 问题</b> ；逻辑判据全过； <b>可访问性硬检查</b> （3 倍字号五档不破版 + 可点范围 $\geq 44$ + 不封顶）全过；其余 <b>128 份</b> 界面判据全过 |
| "线上跑的就是代码这一版" | 漂移检查：网页产物指纹与仓库一致；三台用户盒子跑的程序版本与仓库同一份                                                                   |
| 真机（公网）        | 对外服务 200；浏览器探针实测：页面打得开、 <b>它自己建立的数据连接通</b> （1 条连接、202 帧、登录凭证自动续期）；并人工看了截图确认界面正常                       |
| 文档检查          | 199 份文档里的相互引用 <b>全部指向存在的文件</b> （没有悬空引用）                                                               |
| 开机完整性         | 220 个"开机必读"文件有哈希清单（root 只读）： <b>被改过一个字节就拒绝启动</b>                                                      |

怎么读这些数字：**1513** 不是"跑了 1513 次断言"，而是 1513 条"这件事应该是什么样"的判据；而且每条判据都要求能被"改坏代码"证明它是有效的（第 4 节的第 3 条）。

## 六、安全考虑

| # | 防线             | 具体做法                                                                                                               |
|---|----------------|--------------------------------------------------------------------------------------------------------------------|
| 1 | 助手只能提出申请，签字的是人 | 要改动运行中的系统（装东西、改配置），助手把命令准备好，由人按键执行。理由是：真正的危险不是"它干一次坏事"，而是"坏事能不能留下来"——比如它读到一段看不见的网页内容，被诱导写进自己的配置，从此每次开机都读它，而表面上一切正常 |
| 2 | 一人一台盒子         | 容器隔离 + 只读代码层 + 资源上限；跨用户看不到对方任何东西                                                                                   |
| 3 | 钥匙不出盒子         | 盒内特权小程序握钥匙，助手只有占位符；日志和回复里永不出现钥匙                                                                                    |
| 4 | 小程序：声明 ≠ 已给    | 小程序说"我想要联网 / 想用 AI"只是它想要；用户第一次打开时一张弹窗一次问完，点过头的才生效；数据天生就有存储，但每格独立、跨格读不到、随时能一键清空                                     |
| 5 | 上网留痕           | 助手能访问网络，但去过哪、多少次都落账（不记录内容），可查                                                                                      |
| 6 | 开机完整性清单        | 决定"它下次开机读什么"的文件（人格、规范、能力清单），哈希写在 root 只读清单里，动过就拒绝启动；每次重建都留痕                                                        |
| 7 | 删除要二次确认        | 删之前先把它要带走的東西（聊天 / 数据 / 经验）逐条摆出来再问一次；服务端是软删除，还能拿回来                                                                  |
| 8 | 页面不许说假话        | 做不到的事不许画按钮；拿不到的事实不许猜（"不知道"就说不知道）。这条被当成缺陷来管，有判据盯着                                                                   |

## 七、创新点与自我评价

| 评委关注点 | 本作品对应的东西                                                                                                                 |
|-------|--------------------------------------------------------------------------------------------------------------------------|
| 技术性   | 自己实现的调度内核（可收口的轮次、落盘账本、崩溃后接管）· 自建"一人一台盒子"的容器编排 · 流式语音识别 · 小程序沙箱（只读挂载 + 逐格数据库 + 权限弹窗 + 上网留痕）· <b>1513 条判据 + 变异验证</b> 的工程方法 |
| 智能性   | 大模型不只是聊天：它会 <b>写代码造小程序</b> 、长期记住用户的事、把长活放到后台自己办完再汇报；下一版还要加"先说懂、再问一句、才发出去"的理解层                                            |
| 实用性   | 使用者很宽：从八九岁的孩子到六七十岁的老人，到在手机/平板上移动办公的人 —— <b>会说话就能用</b> ；一句话就能办事；桌面上的东西看得见、摸得着、能撤；已经在真实环境运行，不是演示品                          |
| 安全性   | 第六节的八条防线；其中最硬的三条是"助手只能提出申请、人按按钮"、"钥匙不出盒子"、"开机完整性清单"                                                                      |

## 八、不足与改进方向

- **语音输出还没接通手机：**网页上已经能用设备自带的声音念出来，手机（安卓包）还不能；正在接通云端语音合成。
- **登录还靠临时验证码：**正式短信通道还没接，这是"已知并接受"的临时状态，不是最终形态。
- **性能还没优化：**目前策略是"先功能、后性能"，所以内存占用、并发上限都还比较宽松。
- **"助手卡住"还有一类情况不能自动收尾：**进程死掉能自动收场，但"进程活着却卡在不动的循环里"目前没有自动收尾（这是为了让长活不被时间上限砍掉而做的取舍，写在设计记录里）。
- **平台只有网页和安卓：**iOS 还没做。

附录 A 现场演示步骤（约 4 分钟）

| 时间   | 演示什么                    | 怎么讲                                      |
|------|-------------------------|------------------------------------------|
| 0:00 | 打开页面 → 登录 → 桌面          | "这是他自己的桌面。每一格是一个小程序，也是它的一条对话。"           |
| 0:30 | 按住麦克风说一句人话（语音输入）        | "我不用打字，说一句就行。"（指屏幕上的字）                   |
| 1:00 | 让它做一个小程序："帮我做一个记账的"     | "它现在真的在写代码。"（指桌面上新长出来的那一格：灰的、转着圈）        |
| 1:45 | 等它做完，那一格自己变回正常颜色，点开用一下  | "做好了它自己变回来，点开就能用。"（记一笔，再打开还是那笔 —— 数据存住了） |
| 2:45 | 让它干一件长一点的活，然后合上页面       | "长活它自己放后台办，办完会主动告诉我。"                    |
| 3:15 | 回到桌面，看"在做"的小圈在转         | "哪一件还在办，桌面上看得见。"                         |
| 3:30 | 说一句含糊的话，演示"它问一句"（下一版功能） | "听不清它会问，问清楚才发出去 —— 不替我乱猜。"               |
| 3:50 | 打开设置里的安全项目              | "他自己的钥匙、他点头才开的权限、还有这份开机清单。"              |

演示提示：现场演示前先进入「设置」里的演练入口 —— 那里是一个空白、临时的对话系统，整条流程真跑，但一句都不会真的发出去，所以可以反复演示，不怕说错、不留垃圾。

附录 B 名词解释（给非技术评委）

| 词                | 在这个作品里是什么意思                                                            |
|------------------|------------------------------------------------------------------------|
| 大模型 / 大语言模型      | 能理解并生成自然语言的 AI 模型。这里它不只用来聊天，主要用来"决定下一步做什么、用什么工具"。                      |
| 容器（盒子）           | 一种轻量的隔离技术：在同一个操作系统里划出一块独立的小空间，里面有自己的一套程序和文件，互相看不见。这里"一人一台盒子"。          |
| 语音识别 / 语音合成      | 前者把说的话变成文字（耳朵），后者把文字念出来（嘴）。                                            |
| 判据 / 负向对照 / 变异验证 | 判据 = 一条"这件事应该是什么样"的自动检查；负向对照 = 先证明这个检查抓得住错误；变异验证 = 故意把规矩改坏，看检查是不是当场变红。 |
| 沙箱               | 把一段程序关起来运行的环境，限制它能碰到什么（这里是：代码只读、数据库一格独立、联网要用户点头）。                      |